

Computation and Boundary Evidence for Scientific Repair: Parity, Structural Wins, and a Domain-Dependent Penalty

Rajarshi Ghoshal

PhD research task submission, Graz University of Technology

rajarshi.ghoshal1@gmail.com

Abstract

A scientific repair can be one line long and still be hard to find, because the code that must change is identified by what it computes rather than by where it sits. We ask whether a repair agent benefits from a prepared, queryable account of that structure. Our representation, *computation and boundary evidence* (CBE), records which expressions produce a returned quantity, which conditions guard them, which public definitions document them, and which calls leave the repository to a named provider. It is built from parsers alone, delivered through three query endpoints beside the agent’s ordinary shell tools, and costs no additional time or tokens. We evaluate it on all 119 tasks of SWE-bench Science with DeepSeek V4.1 Flash, under a matched allowance and three to four replicates per task. CBE solves the same number of held-out attempts as the baseline while generating 13.7% fewer output tokens, and its effect separates by discipline: it gains 22.2 percentage points in mechanics and 9.7 in materials science, but loses 16.7 in biomedical engineering and 9.5 in biology, where the defect is an ordinary software fault dressed in scientific language. Replicates show that about a fifth of tasks change outcome between identical runs, so only differences of this size are measurable at all.

1 Introduction

A rotation applied in the wrong reference frame, a unit system assumed rather than converted, a boundary treated as closed when the data are open: failures in scientific software are often one line wide, and finding that line requires knowing which expression produces the quantity that a public interface promises. An agent working with shell tools alone has to reconstruct that structure from text search. This is possible, but it spends the same budget that the repair itself needs.

We ask whether that reconstruction can be supplied in advance, cheaply, and queried on demand. Our representation, *computation and boundary evidence* (CBE), is prepared once per task by parsers and analyzers: it records computations together with the source spans that anchor their results, conditions and findings that guard them, documented public definitions, and every call that crosses a repository boundary, classified as internal, external with a named provider, or unresolved. The agent reaches it through three endpoints — search, inspection and an optional record — while keeping its usual tools. Preparation makes no model call, and the endpoints add no allowance.

We evaluate this against an unchanged repair baseline on the complete SWE-bench Science benchmark Xu et al. (2026). Our contributions are:

1. **A preparation pipeline that needs no model.** File ranking follows execution evidence and boundary callers; Python is parsed with Tree-sitter and C, C++, Fortran and Cython with Joern code property graphs, with explicit fallback when a frontend is unavailable.
2. **Boundary classification as first-class evidence.** Every call is labelled internal, external with a provider, or unresolved. External providers are treated as interfaces assumed correct, which keeps the search inside the repository instead of guessing at library internals.

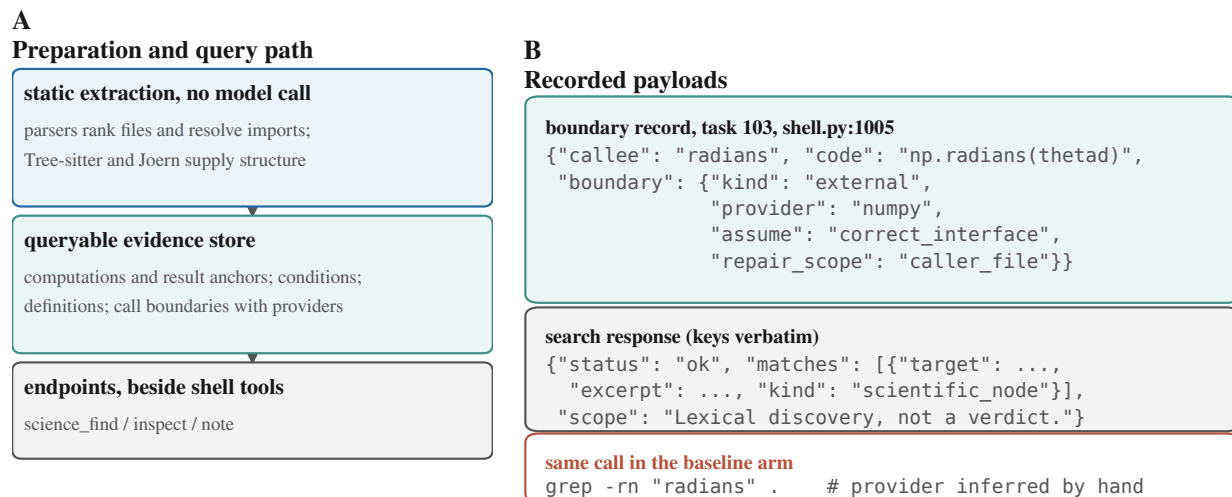


Figure 1: Preparation and query path. The store is built without a model call or code execution, and the payloads shown are recorded from the held-out runs.

3. **A matched evaluation with replicate structure.** We report solve rate, generated tokens, partial credit and per-discipline outcomes over 757 verified attempts, and we quantify how much of the benchmark moves between identical runs.

2 Related work

Linking bug reports to code through intermediate structure beats text similarity, and narrowing the role of retrieved context avoids pollution from loosely related artifacts Niu et al. (2023a;b; 2025); Pan et al. (2025). Representations of scientific code exist as well: code property graphs model control and data flow Yamaguchi et al. (2014), incremental parsers make multi-language parsing routine Brunsfeld et al. (2024), and model-extraction efforts recover equations and semantics from scientific source ML4AI Team (2024); Hein et al. (2020). That work is offline or code-only. CBE differs in delivery: the same structure, distilled to computations and boundaries and handed to a repair agent inside an unchanged harness.

3 What the agent receives

Preparation is a single pass with no model call: files are ranked so that execution evidence and boundary callers precede shallow entry points, and parsers then produce the store. Across the 94 tasks whose prepared bundles are frozen, a task yields a median of 296 scientific objects, and the boundaries recorded during the held-out runs comprise 12,794 external calls, 1,890 unresolved ones and 534 internal ones, with `numpy` alone accounting for 1,554 external cases. The agent reaches the store through `science_find`, `science_inspect` (relationships, definitions, source and contracts) and `science_note`, an optional record that never gates repair. Figure 1 shows the path and a boundary record of the kind the agent receives.

4 Protocol

Thirty development tasks were used for iteration and 89 were locked before the reported runs, so the held-out partition is the only basis for the headline claim. Discipline labels come from the benchmark’s own task inventory, re-extracted and validated against its published counts rather than annotated by us. Both arms use the same model (DeepSeek V4.1 Flash at temperature 0 with a pinned hash seed), image, 1800-second budget and verifier; the verifier runs only after the repair stops and neither container has network. A solve means the official reward, and the private-test pass fraction is kept as partial credit. The held-out partition

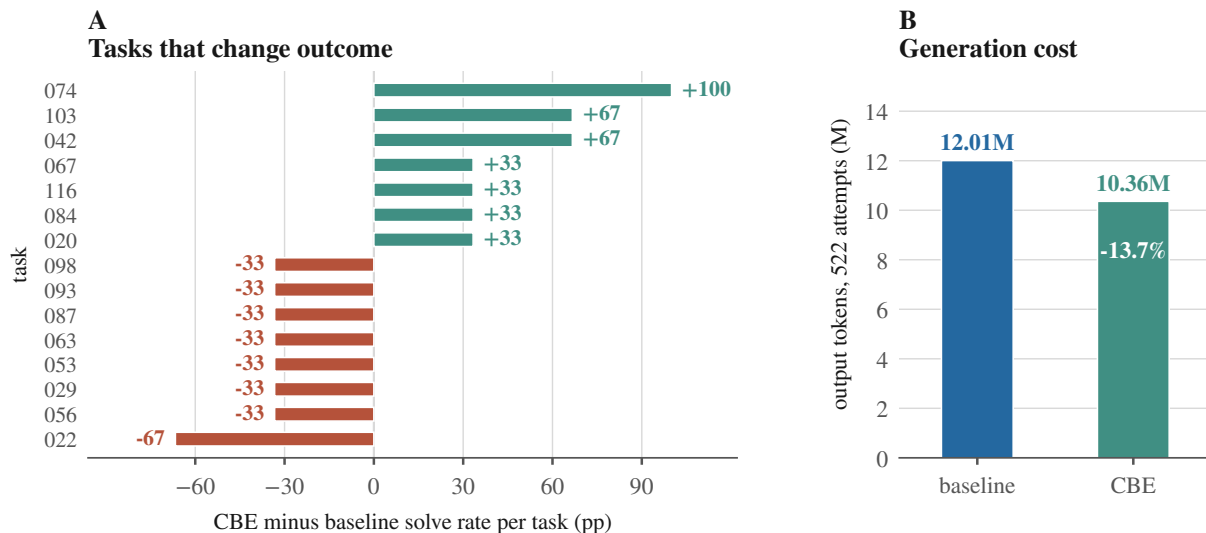


Figure 2: Only fifteen of the 89 held-out tasks change outcome. **A** each of them, as CBE minus baseline solve rate in percentage points: task 074 gains a full point, task 022 loses two thirds of one, and the other 74 tasks do not move. **B** output tokens generated over the 522 verified attempts.

Table 1: Solve counts, cost and partial credit for both arms. The two arms reach the same solve count; CBE spends 13.7% fewer generated tokens and slightly more wall-clock time.

Partition / metric		baseline	CBE
Held-out 89, $k=3$	attempts solved	60/262 (22.9%)	60/260 (23.1%)
	paired tasks (74 ties)	8 better	7 better
	output tokens	12.01M	10.36M (-13.7%)
	private tests passed	2315/3167 (73.1%)	2226/3148 (70.7%)
Development 30, $k=4$	attempts solved	43/117 (36.8%)	44/118 (37.3%)
Development 30, effort	high / low	12/30 / 12/30	11/30 / 13/30
Matched 20, effort	high / low	10/20 / 10/20	8/20 / 10/20

was replicated three times and development four times, and every number below regenerates from verifier receipts through one analysis script.

5 Results

Parity costs fewer generated tokens. CBE solves as many held-out attempts as the baseline (Table 1, Figure 2), and the paired comparison over tasks is flat: 74 tie, seven favour CBE and eight favour the baseline, with a sign test that cannot separate them. Development repeats the pattern. What changes is cost. CBE generates 13.7% fewer output tokens, close to 40,000 per attempt instead of 46,000, spends slightly more input because it still opens files, and takes about a minute more wall-clock time per attempt. Partial credit moves the other way by a small margin, which matches a narrower search: the agent either finds the right computation or stops before it.

Gains and losses separate by discipline. Aggregate parity hides opposing effects (Figure 3). Mechanics gains 22.2 percentage points, from no solves to two of nine attempts, astronomy 12.4 and materials science 9.7, while biomedical engineering loses 16.7 and biology 9.5; chemistry, physics and mathematics are unchanged to the last attempt. Development shows the same shape from the other side, with mathematics gaining three

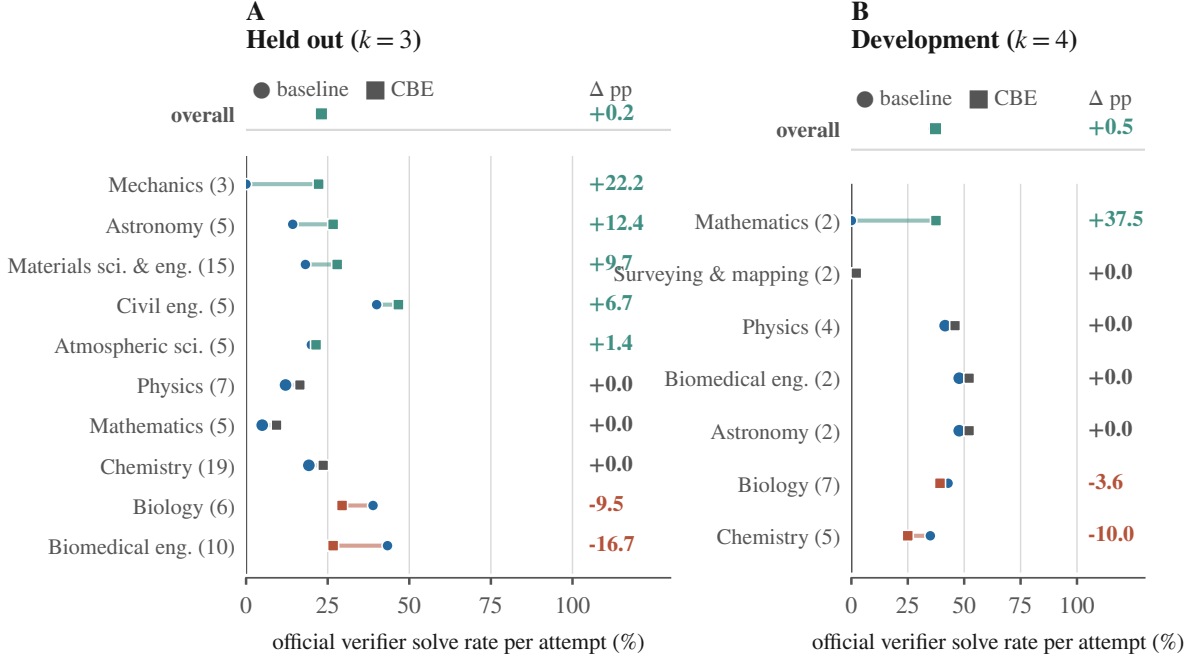


Figure 3: Wins and losses separate by discipline. **A** held-out tasks; **B** development tasks. Each row is one domain, labelled with its task count: circles are the baseline arm, squares CBE, and the right column gives the difference in percentage points. Where the two markers coincide the domain did not move at all.

solves where the baseline had none. The benchmark’s knowledge-ablation flag shows no interaction, and language strata are uninformative because 81 of the 89 held-out tasks are pure Python.

The tasks themselves name the mechanism. CBE gains on tasks 020, 042, 067, 074, 084, 103 and 116, and loses on 022, 029, 053, 056, 063, 087, 093 and 098. Task 103 asks for the stiffness matrices of a symmetric composite laminate; CBE solved it twice in three replicates against no baseline solve, inspecting the ply-angle computation and the library boundaries it calls. Task 022 asks for a volume-to-surface projection in a neuroimaging toolkit, and the comparison inverts: the baseline solved it twice and CBE never did, because the trajectory reasoned about projection mathematics before touching the atlas label mapping the instruction describes. When the defect is a computation, structure locates it; when the defect is ordinary code under scientific prose, structure distracts from it.

Replication bounds what is measurable. Three replicates make the measurement problem visible (Figure 4). Sixty tasks are never solved by the baseline and 61 never by CBE, eleven and ten are solved every time, 18 change outcome between identical runs within one arm, and the arms disagree on 15 tasks in total. A single run would attribute a few percentage points to the method with no basis.

Reduced reasoning effort recovers what high effort lost. On the development partition the baseline leads at high effort and CBE leads at low effort, and on the 20-task subset evaluated at both levels the high-effort deficit disappears once effort is reduced. The two tasks recovered under low effort are the same two that were lost under high effort, which is what one expects if external evidence substitutes for deliberation rather than adding capability. The agent used the endpoints throughout that subset: 61 inspections, 10 searches and 20 optional records across 20 tasks.

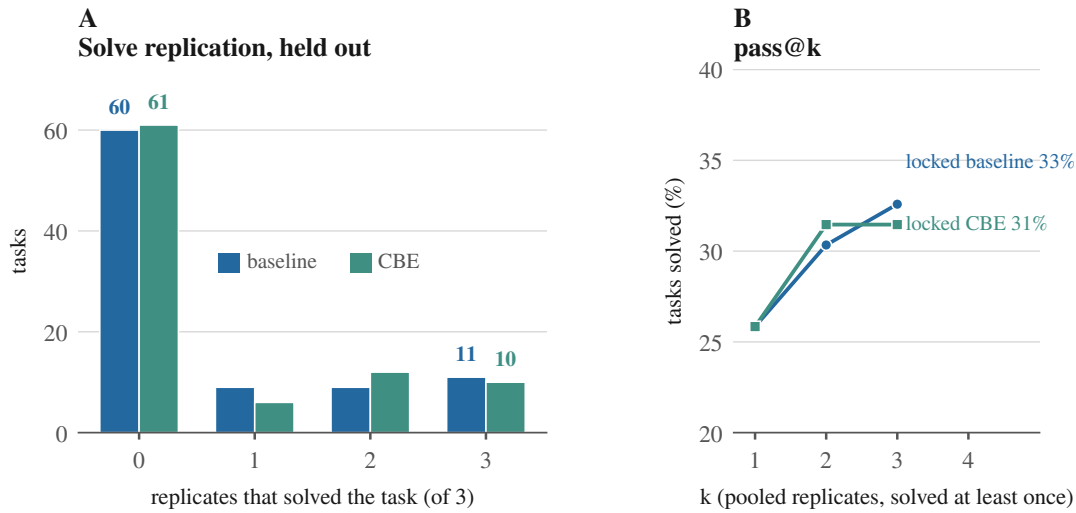


Figure 4: Most tasks never move. **A** held-out tasks by the number of replicate runs that solved them: 60 (baseline) and 61 (CBE) are never solved, 11 and 10 are always solved. **B** share of tasks solved by at least one of k pooled runs, $k \leq 3$. Across arms, eighteen tasks flip inside one arm and fifteen differ between arms.

6 Discussion and conclusion

One reading of these results is that scientific structure is not useful to a repair agent, since average solve rate does not move. The other is that its usefulness is domain-dependent: it helps where the defect is a computation and hurts where the defect is code that merely lives near one. The discipline split, the discordant tasks and the partial-credit shift all point at the second reading.

Four limits bound the claim. One model at one budget produced every number; single-task disciplines are descriptive, and only the aggregate held-out comparison supports inference; unparseable expressions are recorded as unsupported and external calls are labelled rather than verified, so a classification error would silently shrink the search space; and development tasks are optimistic by construction. The design consequence is to offer scientific structure conditionally, with a signal that the defect is computational before an agent commits to reading mathematics.

Statements

AI use. The author designed the representation, the protocol and the analysis, and wrote this report. LLM coding assistants implemented harness plumbing, parsers and figures, and helped revise prose. DeepSeek V4.1 Flash was the only model used for repair attempts; the development partition was the only basis for iteration. No prior research of the author is claimed as the basis of this method.

Reproducibility. The 757 verified attempts (774 scheduled) total 115/379 for the baseline and 117/378 for CBE. Numbers regenerate from `analysis/paper_facts.py` into `results/paper-facts.json`, and figures render from frozen receipts through `paper/figures/make_all.py`, which asserts its layout before writing. Code, receipts, exact task IDs and a reproduction README are in the submission repository recorded in `README.md`.

References

- Z. Xu, J. Lu, Y. Zheng, Y. Wang, and X. Qiu. SWE-bench Science: can coding agents resolve engineering tasks in science? *arXiv preprint arXiv:2608.19799v2*, 2026.
- C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- F. Niu, C. Mayr-Dorn, W. K. G. Assunção, L. Huang, J. Ge, B. Luo, and A. Egyed. The ABLoTS approach for bug localization: is it replicable and generalizable? In *Proc. 20th International Conference on Mining Software Repositories (MSR)*, pp. 152–162, 2023.
- F. Niu, W. K. G. Assunção, L. Huang, C. Mayr-Dorn, J. Ge, B. Luo, and A. Egyed. RAT: a refactoring-aware traceability model for bug localization. In *Proc. 45th International Conference on Software Engineering (ICSE)*, 2023.
- F. Niu, L. Huang, C. Mayr-Dorn, and A. Egyed. TVR: automotive system requirement traceability validation and recovery through retrieval-augmented generation. *arXiv preprint arXiv:2504.15427*, 2025.
- X. Pan, F. Niu, L. Huang, and A. Egyed. Requirements coverage-guided minimization for natural language test cases. *arXiv preprint arXiv:2505.20004*, 2025.
- ML4AI Team. GroMEt: a unified intermediate representation for scientific models and code. DARPA ASKEM / SKEMA consortium, 2024.
- P. Hein et al. AutoMATES: automated model assembly from text, equations and software. *arXiv preprint arXiv:2001.07295*, 2020.
- F. Yamaguchi, N. Golde, D. Arp, and K. Rieck. Modeling and discovering vulnerabilities with code property graphs. In *IEEE Symposium on Security and Privacy (S&P)*, 2014.
- M. Brunsfeld et al. Tree-sitter: an incremental parsing system for programming tools. <https://tree-sitter.github.io>, 2024.
- DeepSeek. DeepSeek V4.1 Flash API documentation: reasoning-effort parameter. <https://api-docs.deepseek.com>, 2026.